Rivive Shield: Project Proposal

Software Development Methods | Team of 5 | Leader: Usama Shmasneh
4 November 2026. Lecturer decision: project approved, as reported by Usama Shmasneh; approval date and conditions not provided.

1. User and problem

The intended users are students operating and evaluating an isolated demonstration application. A failure alert alone does not explain the cause, establish a safe repair or prove that normal behavior has returned. Rivive Shield will link these steps in one inspectable workflow. The primary operator reviews evidence and approves a bounded repair; participants exercise the booking application using fictitious data.

2. Today’s workaround

The working assumption is that an operator manually checks service status and logs, changes the application state and repeats tests. Target-user validation is not reported here. A planned operator walkthrough will record the steps and difficulties encountered in a recent debugging task.

3. Competitors and alternatives

Alternatives to compare are manual debugging, a monitoring dashboard with a written recovery checklist, and a predefined repair script. Prometheus/Alertmanager provides monitoring and alert notification; Ansible playbooks provide predefined automation. Official-source links and the comparison are in the first-submission report. User-study results are not included in this submission. We do not claim that these alternatives lack relevant capabilities or that Rivive Shield is a new commercial category.

4. Why switch?

Our hypothesis is that students will find one evidence-to-repair-to-verification workflow easier to follow and evaluate than separate tools. We will test this with a short operator task. The benefit is a hypothesis, not an established competitive advantage.

5. Proposed MVP: three user stories

1. **Participant:** attempt a permitted challenge against the real demo and see a result based on observed effects.
2. **Operator, central story:** inspect a detected incident, review a supported diagnosis and repair plan, approve it, and verify recovery.
3. **Operator:** retest the logical fault and a variant, confirm a legitimate booking still works, and inspect the saved evidence.

Scope: one booking application, one controlled service-stop scenario, one proposed booking-rule violation, and predefined repairs. Excluded: arbitrary bug fixing, autonomous unrestricted repair, production systems, device tracking and identification of a real attacker. The proposed logical scenario accepts a later overlapping booking; the repair preserves the earlier booking, marks only the conflicting later record inactive and activates atomic overlap validation. The detailed scope and repair rule are the proposed design; project approval does not establish separate approval of each detail.

6. Goal, objective and success metric

Goal: make a supported incident understandable, recoverable and independently verifiable. **Objective:** complete the central workflow from supported evidence to an approved repair, verified recovery and a saved incident record. **Metric:** complete successful trials divided by all scheduled valid supported trials, plus approval-to-verification time. A successful trial requires preserved legitimate data, a healthy service, a successful normal booking and a saved incident record. Trial conditions and target values will be fixed before evaluation; no test results are reported here.

7. Main risks and feasibility

Risks include excessive scope, unsupported AI conclusions, unsafe repair and unavailable free AI access. Mitigations are two bounded scenarios, evidence-linked explanations, an allowlist of repair commands, human approval and independent retesting. Use only fictitious data in an isolated environment. C++, SQLite and a minimal web interface are intended; hosting and AI-provider feasibility remain to be checked. Usama owns the C++ incident state machine alongside leadership, database, external AI connection, Oracle and GitHub Actions.

AI-agent role: useful for selecting read-only diagnostic checks and explaining a supported plan; deterministic code validates, executes and verifies only approved actions. It will not receive the challenge ID as the answer, run arbitrary commands or write arbitrary SQL.

Team: Usama Shmasneh; Samy mansour; Ameen Abd Allader; Shehab Shehab; Yazan Ganem. The team responsibilities and planned handoffs are detailed in the report. A one-page PDF accompanies this proposal.